

SERVING DE MODÈLES OPEN-WEIGHT

Qu'est-ce qui tourne sur ma machine, et pourquoi à cette vitesse-là ?

Le cours complet — modules 0 à 11

Inference & serving · Pas de training, pas de finetuning

Vue d'ensemble du cours

00	Cadrage : ce qu'est le serving, et quel outil existe pour quoi
01	Hello serving : un serveur qui tourne, une API qui répond
02	Lire un modèle sur Hugging Face et décider s'il convient
03	Budget mémoire et KV cache — « est-ce que ça rentre ? »
04	Prefill, decode, batching, caching — « pourquoi c'est lent ? »
05	Compression : quantization, pruning — « faire rentrer plus gros »
06	Génération et sampling — contrôler la sortie
07	llama.cpp — l'engine single-user
08	Benchmarking et observabilité — mesurer au lieu de croire
09	vLLM — l'engine multi-tenant
10	Couche client : chat templates, tool calling, Open WebUI
11	Synthèse : l'arbre de décision

00

Cadrage et panorama

Où se situe le serving

Le *training*, c'est l'école : coûteux, lent, fait **une fois**. Le *serving*, c'est le travail quotidien : peu coûteux par requête, mais répété **des milliards de fois** . C'est pour ça qu'on l'optimise autant.

Pretraining

Des milliers de GPU pendant des mois.
Le modèle apprend la langue.

Post-training

SFT, RLHF, DPO : on lui apprend à obéir et à converser.

Poids publiés

Un dossier de fichiers sur Hugging Face. C'est notre point de départ.

Serving — nous

Charger ces poids en mémoire, répondre aux requêtes, vite et pas cher.

Conséquence directe : pendant tout le cours, **les poids ne changent jamais** . On ne peut jouer que sur la façon de les stocker, de les déplacer en mémoire et d'organiser le calcul.

Les trois quantités à optimiser

QUANTITÉ	CE QU'ON MESURE	UNITÉ	QUI S'EN SOUCIE
Latence d'attente	Temps avant le premier mot — TTFT	secondes	L'utilisateur qui regarde l'écran
Vitesse de frappe	Mots par seconde ensuite — tokens/s	tok/s	Le même utilisateur, en train de lire
Throughput	Tokens produits par le serveur, tous clients confondus	tok/s total	Celui qui paie la facture GPU

Le compromis : faire attendre les requêtes pour les traiter par paquets augmente le throughput total... et allonge l'attente de chacun. Aucun réglage n'améliore les trois à la fois : tout le cours navigue ce compromis.

Deux régimes de charge

	LOCAL · SINGLE-USER	PRODUCTION · MULTI-TENANT
Scénario	Vous, sur votre laptop, un GPU	500 employés sur un chatbot interne
Métrique reine	Latence perçue : tok/s pour <i>moi</i>	Throughput total et coût / M tokens
Hardware	GPU consumer, CPU+RAM, Mac	H100 / H200 / B200, multi-GPU
Contrainte dominante	La VRAM : le modèle doit rentrer	Le coût : le GPU doit être saturé
On accepte de sacrifier	Le throughput, la qualité du quant	La latence individuelle, la flexibilité
Engine représentatif	llama.cpp	vLLM

llama.cpp et vLLM ne sont donc **pas concurrents** : ils optimisent des fonctions objectif différentes. Comparer leurs tokens/s sans dire à quelle concurrence n'a aucun sens.

Les engines : qui pour quoi

ENGINE	TECHNIQUE	POURQUOI ON LE CHOISIRAIT
llama.cpp	C++, format GGUF, CPU+GPU hybride	Tourne partout, même sans assez de VRAM : un binaire, aucune dépendance
vLLM	Python/CUDA, PagedAttention	Standard de fait en production : sert des dizaines de clients sur un même GPU
SGLang	RadixAttention	Concurrent direct de vLLM, fort sur les workloads agentiques
TensorRT-LLM	NVIDIA, compilation dédiée	Perf maximale, au prix d'un build par modèle et par config
MLX / mlx-lm	Apple Silicon, unified memory	Le choix naturel sur un Mac M-series

Existent aussi : ExLlamaV2/V3 (GPU-only, quants EXL), ik_llama.cpp (fork perf hybride), Hugging Face TGI (historique, moins central aujourd'hui).

Engines, wrappers, providers

Un **engine** est le moteur : il fait tourner le modèle. Un **wrapper** est la carrosserie : même moteur, meilleure ergonomie. Un **provider** fait tourner ces engines à votre place et vous facture les tokens.

Engine

llama.cpp, vLLM, SGLang

Vous exposez tous les réglages : placement mémoire, contexte, batch, quant.

Wrapper

Ollama, LM Studio, KoboldCpp, Jan

Ajoute : catalogue de modèles, téléchargement auto, presets, interface. **Cache** : les flags — donc le contrôle.

Provider

OpenRouter, Groq, Together, Fireworks

Font tourner vLLM ou SGLang à grande échelle. Comprendre le serving, c'est comprendre leur facture.

« Ollama vs llama.cpp » n'est donc pas une comparaison d'engines : c'est une comparaison d'ergonomies.

À MARTELER

« Il n'y a pas de meilleur engine. Il y a un engine par régime de charge. »

Corollaire pratique : la première question à poser devant un choix d'outil n'est jamais « lequel est le plus rapide ? » mais « combien d'utilisateurs simultanés, et sur quel hardware ? ».

01

Hello serving

Le vocabulaire minimal

TERME	DÉFINITION OPÉRATIONNELLE	ORDRE DE GRANDEUR
token	L'unité que le modèle lit et écrit : un morceau de mot	1 token $\approx$ 4 caractères $\approx$ 0,75 mot
context	La fenêtre de travail : prompt + réponse, tout compris	4k à 128k tokens
poids	Les nombres appris, figés, chargés en mémoire au démarrage	4 à 140 GB de fichiers
VRAM	La mémoire du GPU : rapide, et petite	8 à 141 GB selon la carte
quant	Version compressée des poids (module 5)	Q4 $\approx$ 4× plus petit que bf16

Exemple de tokenisation : "Le serving, c'est fascinant" → ["Le", " serv", "ing", ",,", " c", "'", "est", " fasc", "inant"] — 9 tokens pour 4 mots. Un mot rare coûte plus de tokens qu'un mot courant, et le français coûte plus que l'anglais.

Ce que fait le serveur au démarrage

```
llama-server -hf bartowski/Qwen2.5-7B-Instruct-GGUF :Q4_K_M -c 8192 -ngl 99
```

- 1. Télécharge** le fichier GGUF depuis Hugging Face et le met en cache local (~/ .cache) — une seule fois.
 - 2. Charge les poids** en mémoire : -ngl 99 demande de tout mettre sur le GPU ; ce qui ne rentre pas reste en RAM.
 - 3. Réserve le KV cache** pour 8192 tokens de conversation. Cette réservation est fixe, faite d'avance, et coûte de la VRAM (module 3).
 - 4. Lit le chat template** embarqué dans le GGUF : la façon de présenter les messages à ce modèle précis (module 10).
 - 5. Expose une API HTTP** sur localhost:8080 , compatible OpenAI, plus une petite interface web.
- À lire dans les logs :** la répartition des tensors par device et la taille allouée du KV cache. Ce sont les deux chiffres qui expliquent la vitesse.

Même geste, hypothèses opposées

LLAMA.CPP

```
llama-server -hf <repo>:Q4_K_M -c 8192
```

VLLM

```
vllm serve <repo> --max-model-len 8192
```

	LLAMA.CPP	VLLM
Temps de démarrage	Quelques secondes	Dizaines de secondes à minutes (compilation, CUDA graphs)
Format attendu	GGUF, un fichier	safetensors, AWQ, GPTQ, FP8...
Si ça ne rentre pas	Déborde en RAM et continue, plus lentement	Échec au démarrage : il faut réduire le contexte
Mémoire	Alloue ce qu'il faut	Préempte ~90 % de la VRAM d'emblée

L'API OpenAI-compatible

OpenAI a défini ce format en premier ; il s'est imposé comme standard de fait. Tous les engines du cours l'implémentent, donc **changer d'engine ne change pas votre code : seulement l'URL** .

ENDPOINT	À QUOI IL SERT	STATUT
GET /v1/models	Quel(s) modèle(s) ce serveur expose, sous quel nom	Utile pour le debug
POST /v1/chat/completions	Une conversation : liste de messages → réponse	Celui qu'on utilise
POST /v1/completions	Complétion de texte brut, sans rôles ni template	En voie de disparition
POST /v1/embeddings	Vecteurs pour la recherche sémantique / RAG	Autres modèles (module 2)

Pourquoi /completions disparaît : sans rôles, c'est à vous d'écrire les balises spéciales du modèle à la main — source d'erreurs silencieuses.

Ce que vous envoyez, ce que le modèle voit

Votre requête — propre

```
"messages": [  
  {"role": "system",  
   "content": "Tu es concis."},  
  {"role": "user",  
   "content": "Bonjour !" }  
]
```

system — les consignes permanentes ; **user** — la question ; **assistant** — les réponses passées ; **tool** — le résultat d'un outil appelé.

Ce que le modèle reçoit — balisé

```
<|im_start|> system  
Tu es concis.  
<|im_end|><|im_start|> user  
Bonjour !  
  
<|im_end|><|im_start|> assistant
```

Ces balises sont propres à chaque famille de modèle. Le serveur les ajoute via le **chat template** : s'il se trompe, le modèle répond mal sans qu'aucune erreur ne s'affiche.

Streaming : la même réponse, autrement livrée

Avec `stream: true`, le serveur envoie chaque token dès qu'il est produit, sur une connexion HTTP maintenue ouverte (SSE, Server-Sent Events).

```
data: {"choices":[{"delta":{"content":"Bon"}}]}
data: {"choices":[{"delta":{"content":"jour"}}]}
data: {"choices":[{"delta":{"content":" !"}}]}
data: [DONE]
```

Ce que ça change : rien à la vitesse totale. L'utilisateur commence à lire après le premier token au lieu d'attendre le dernier — l'attente devient supportable.

Deux temps à ne pas confondre : le **TTFT** (avant le premier mot, dû au prefill) et la **vitesse de frappe** ensuite (tok/s, due au decode). Modules 4 et 8.

Les paramètres qu'on touche vraiment

PARAMÈTRE	EFFET	PIÈGE COURANT
temperature	0 : toujours le mot le plus probable. 1 : de la place pour la surprise	Un modèle reasoning à température 0 boucle souvent
max_tokens	Plafond de longueur de la réponse	Trop bas : la réponse est coupée net, sans erreur
stream	Livraison token par token	Le comptage de tokens arrive à la fin, pas au début
seed	Fixe le hasard, pour reproduire une sortie	Ne suffit pas : le batching change les résultats (module 6)
stop	Chaînes qui interrompent la génération	Oublié : le modèle se met à jouer les deux rôles

À retenir dès maintenant : les valeurs recommandées par l'auteur du modèle ne sont **pas** appliquées automatiquement par les serveurs. Beaucoup de « ce modèle est nul » sont en fait des problèmes de sampling (module 6).

À vous de jouer

```
$ llama-server -hf bartowski/Qwen2.5-1.5B-Instruct-GGUF:Q4_K_M -c 4096 $ curl localhost:8080/v1/models # quel  
nom expose-t-il ?$ python3 -c "  
from openai import OpenAI  
client = OpenAI(base_url='http://localhost:8080/v1', api_key='none')  
for c in client.chat.completions.create(model='qwen', stream=True,  
    messages=[{'role':'user','content':'Explique le KV cache en une phrase.'}]):  
    print(c.choices[0].delta.content or '', end='', flush=True)"
```

Observez 1 — combien de secondes avant le premier mot ?

Observez 2 — à quelle vitesse le texte défile ensuite ?

Observez 3 — que dit le serveur dans son terminal pendant ce temps ?

TROIS QUESTIONS POUR LA SUITE DU COURS

1 Pourquoi ce modèle-là et pas un plus gros ?

→ modules 2, 3 et 5 : taille des poids, mémoire disponible, compression

2 Pourquoi 3 secondes d'attente, puis 40 tokens/s ?

→ module 4 : le prefill est borné par le calcul, le decode par la bande passante mémoire

3 Que se passe-t-il si dix personnes se connectent ?

→ modules 4 et 9 : batching, KV cache partagé, et le choix de vLLM

À MARTELER

« Tout ce qui suit est en dessous de cette API. L'API ne change pas ; ce qui change, c'est ce qu'il y a derrière. »

Vous avez déjà tout ce qu'il faut pour construire une application. Le reste du cours sert à ce que cette application soit rapide, tienne la charge, et ne coûte pas trois fois trop cher.

02

Lire un modèle sur Hugging Face

Trois questions, une décision

Vous tapez « Llama » sur Hugging Face : deux cents résultats, dont des repos officiels, des quants tiers, des merges et des finetunes abandonnés. Comment trancher ?

1. Est-ce le bon modèle ?

Bonne famille, bon stade de post-training, bonne taille pour mon hardware, bonnes capacités annoncées.

2. Est-ce le bon fichier ?

Bon format pour mon engine, bon niveau de compression, quant produit sérieusement.

3. Ai-je le droit de l'utiliser ?

Licence du modèle, restrictions d'usage, repo gated — à vérifier *avant* de construire dessus.

Chercher sur le Hub

Les signaux, et ce qu'ils valent

Trending — popularité des derniers jours. Repère les sorties, pas la qualité.

Downloads — usage réel cumulé. Le signal le plus honnête, mais en retard sur l'actualité.

Likes — appréciation communautaire. Sensible à la hype.

Filtres utiles : **task**, **library** (GGUF, safetensors), **language**, **license**, **size** . Et dans les Spaces : les calculateurs de VRAM — pratiques comme vérification, jamais comme vérité (module 3).

Repo officiel ou quant tiers ?

Qwen / Qwen3-8B — l'organisation qui a entraîné le modèle : poids d'origine, model card de référence.

bartowski / Qwen3-8B-GGUF — un quantizer tiers : même modèle, recompressé par quelqu'un d'autre.

Quantizers de confiance : unsloth, bartowski, mradermacher, RedHatAI.

La model card, dans l'ordre

CE QU'ON CHERCHE	POURQUOI	SIGNAL D'ALARME
Architecture & taille	Détermine si ça rentre en mémoire	Aucune mention : repo à fuir
Stade de post-training	base, instruct ou reasoning : change tout l'usage	Nom ambigu, pas de mention « instruct »
Context length	Plafond de la fenêtre de travail	1M annoncé sans résultat long-context
Sampling recommandé	À passer à la main au serveur (module 6)	Absent : vous devinerez
Langues	Multilinguisme réel vs annoncé	« 100+ langues » sans détail
Licence	Votre droit d'usage, y compris commercial	« other » sans fichier de licence

Les benchmarks affichés dans une model card sont **auto-déclarés**. On y revient en fin de module.

Anatomie d'un repo

FICHIER	CONTENU	CE QUI CASSE S'IL EST FAUX
config.json	Architecture, layers, heads, contexte max	Le modèle ne se charge pas du tout
tokenizer_config.json	Le chat_template et les special tokens	Il répond mal, sans aucune erreur
tokenizer.json	Le vocabulaire (BPE, SentencePiece...)	Texte illisible en sortie
generation_config.json	Sampling recommandé par l'auteur	Ignoré par la plupart des serveurs
model-000xx-of-000yy .safetensors	Les poids, découpés en shards de ~5 GB	Un shard manquant : chargement impossible
*.gguf	Tout ce qui précède, dans un seul fichier	Format alternatif : llama.cpp uniquement

Pourquoi le découpage en shards ? Parce qu'un fichier de 140 GB est ingérable : téléchargement reprenable, chargement parallèle, et un `index.json` dit quel tenseur vit dans quel morceau.

Lire un config.json

```
{
  "architectures": ["Qwen3ForCausalLM"],
  "hidden_size": 3584,
  "num_hidden_layers": 28,
  "num_attention_heads": 28,
  "num_key_value_heads": 4,
  "max_position_embeddings": 32768,
  "torch_dtype": "bfloat16",
  "rope_scaling": null
}
```

architectures — la famille. *ForCausalLM* = un modèle qui génère du texte, le cas normal.

num_hidden_layers — 28 blocs empilés. Plus de blocs = plus de mémoire et plus de latence.

num_attention_heads vs **num_key_value_heads** — 28 contre 4 : c'est du **GQA**. Le KV cache est divisé par 7 par rapport à une attention classique. Ce seul rapport change tout au module 3.

max_position_embeddings — 32 768 tokens de contexte maximum *en théorie*.

torch_dtype — les poids publiés sont en bf16, donc 2 octets par paramètre.

Trois formats de poids

.bin — legacy

Pickle Python : charger le fichier pouvait **exécuter du code** — comme une macro piégée dans un Word. À ne plus utiliser.

safetensors

Un header JSON, puis les nombres bruts. Aucun code exécuté, et lecture **zero-copy** par mmap : le système lit les octets directement sans les recopier. C'est le standard côté GPU.

GGUF

Poids + métadonnées + tokenizer + chat template, dans un fichier unique et autonome. C'est le format de llama.cpp : rien d'autre à télécharger.

Conséquence pratique : le format décide de l'engine, et réciproquement. Un GGUF ne se sert pas sous vLLM en production ; un safetensors bf16 ne se sert pas sous llama.cpp sans conversion. Ce n'est pas un choix indépendant (module 5).

Calculer la taille des poids

taille des poids = nombre de paramètres × octets par paramètre

Un modèle 7B en bf16 : **7 000 000 000 × 2 octets = 14 GB**. Le même en Q4 (un demi-octet par paramètre) : **≈ 4 GB** — il passe alors sur une carte 8 GB, ce que 14 GB ne permettait pas.

MODÈLE	BF16 — 2 O	Q8 — 1 O	Q4 — 0,5 O	TIENT SUR
7B	14 GB	7 GB	4 GB	Une RTX 4060 8 GB, en Q4
30B	60 GB	30 GB	17 GB	Une RTX 4090 24 GB, en Q4
70B	140 GB	70 GB	40 GB	Deux GPU 24 GB, ou de l'offload

Attention : ce n'est que le coût des **poids**. Il faudra encore loger le KV cache et les activations — c'est tout l'objet du module 3.

Ce que la taille ne dit pas

Ce que le nombre de paramètres prédit bien

La **mémoire** nécessaire — calcul exact, vu à la slide précédente.

Le **coût** d'hébergement, qui suit la mémoire.

La **vitesse** de génération — pour un modèle dense (module 4).

Ce qu'il prédit mal

La **qualité** . Un 8B récent bat régulièrement un 70B de deux ans plus vieux : meilleures données, meilleur post-training, meilleure architecture.

La **spécialisation** . Un petit modèle entraîné pour le code peut battre un généraliste trois fois plus gros sur votre tâche.

Règle pratique : choisissez d'abord la génération et la spécialisation du modèle, puis la plus grosse taille que votre mémoire tolère — jamais l'inverse.

Dense vs MoE

Dense

Chaque paramètre participe à chaque token. Simple, prévisible : la mémoire et la vitesse découlent directement de la taille.

Exemple concret : 128 experts stockés, 8 activés par token. Il faut loger les 128 en mémoire, mais on ne calcule que sur 8 : **gros en mémoire, rapide en calcul** . On le vérifie dans `config.json` avec `num_local_experts` et `num_experts_per_tok`.

Pourquoi cette architecture existe : gagner de la capacité sans payer le calcul de tous les paramètres à chaque token. Et conséquence pour nous : les experts endormis peuvent vivre en RAM au lieu de la VRAM — c'est la ruse centrale du module 5.

MoE — Mixture of Experts

Le réseau contient de nombreux « experts » spécialisés. Un petit **router** choisit, pour chaque token, lesquels consulter — les autres restent inactifs.

02 · LIRE UN MODÈLE

30B-A3B

30B = total params, ce qu'il faut **stocker**. **A3B** = active params, ce qui détermine la **vitesse** de génération.

Comme un hôpital de 30 milliards de spécialistes : il faut tous les loger, mais chaque patient n'en consulte que 3 milliards. Le loyer suit l'effectif complet ; la durée de la consultation, non.

Conséquence à retenir : un 30B-A3B occupe la mémoire d'un 30B et génère à la vitesse d'un 3B. Exemples : Qwen3-30B-A3B, GLM-4.5-Air, gpt-oss-120b.

Base, instruct, reasoning

STADE	CE QU'IL SAIT FAIRE	SUR « QUELLE EST LA CAPITALE DE LA FRANCE ? »
base	Prédire la suite d'un texte. Rien d'autre.	Enchaîne « Quelle est la capitale de l'Italie ? » — il continue la liste au lieu de répondre
instruct / chat	Suivre une consigne, tenir une conversation	« Paris. »
reasoning / thinking	Réfléchir à voix haute avant de répondre	Produit un bloc de réflexion, puis « Paris. » — plus lent, plus cher en tokens

Hybrid thinking : un même modèle avec un interrupteur (/think, enable_thinking). Réfléchi quand il faut, direct sinon.

Piège des distills : DeepSeek-R1-Distill-Qwen-32B n'est pas un DeepSeek : c'est un Qwen entraîné à imiter R1. Le nom ne dit pas l'architecture.

Les sigles d'attention, traduits

Pour générer, le modèle garde en mémoire un résumé de tout ce qui précède : le **KV cache** . Il grossit avec la longueur de conversation et avec le nombre d'utilisateurs. Ces sigles disent à quelle vitesse il grossit.

SIGLE	CE QUE ÇA FAIT, EN UNE PHRASE	EFFET SUR LA MÉMOIRE
MHA	L'attention historique : chaque tête garde son propre résumé	Référence — la plus coûteuse
GQA	Plusieurs têtes partagent un même résumé	Divisée par le ratio (souvent 4 à 8×)
MLA	Le résumé est compressé avant d'être stocké (DeepSeek)	Réduite d'un ordre de grandeur
Sliding window	Le modèle ne regarde que les N derniers tokens (Gemma, Mistral)	Plafonnée, quelle que soit la longueur
RoPE / YaRN	Étire un modèle au-delà du contexte pour lequel il a été entraîné	Contexte plus long : qualité en baisse

Le contexte *annoncé* n'est pas le contexte *utile* : un modèle annoncé à 1M de tokens peut perdre le fil bien avant. Le module 3 fait les calculs ; le module 2 vous apprend juste à repérer l'information.

Tous les modèles ne génèrent pas du texte

TYPE	À QUOI ÇA SERT	CÔTÉ SERVING
CausalLM (decoder-only)	Générer du texte — le cas général de ce cours	Support universel
Embedding models	Transformer un texte en vecteur, pour la recherche / RAG	vLLM les sert aussi
Rerankers	Classer des documents par pertinence pour une requête	Petits, peu coûteux
VLM (vision + texte)	Lire une image et en parler	Support très inégal, toujours le dernier à arriver
Seq2Seq, encoder-only	Traduction (T5), classification (BERT-like)	Hors périmètre du cours

Côté capacités annoncées, trois choses à chercher dans une model card : **tool calling** (le modèle peut répondre « appelle get_weather (Paris) » au lieu de deviner), **structured output** (forcer du JSON valide) et le **long context** réellement mesuré.

Open weight ≠ open source

« Open weight » veut dire : les poids sont téléchargeables. Cela ne dit rien du droit d'usage, ni du code d'entraînement, ni des données. Quasi aucun modèle n'est « open source » au sens de l'OSI.

LICENCE	EXEMPLES	CE QUI PEUT VOUS BLOQUER
Apache 2.0 / MIT	Qwen, Mistral, DeepSeek	Rien : usage commercial libre
Llama Community License	Llama 3.x, 4	Au-delà de 700 M d'utilisateurs mensuels, autorisation Meta requise ; clause de naming
Gemma Terms of Use	Gemma 2, 3	Use policy contraignante, applicable en aval
Non-commerciale	Certains finetunes de recherche	Interdit en production : vérifier avant de prototyper

Repos **gated** : il faut accepter les conditions sur le site, puis fournir un HF_TOKEN au serveur. Et pour un quant tiers, deux licences comptent : celle du modèle et celle du dataset de calibration.

Cinq benchmarks, cinq capacités

Un benchmark est un examen standardisé : mêmes questions pour tous les modèles, score comparable. Chacun ne teste qu'une capacité.

BENCHMARK	CAPACITÉ MESURÉE	FORMAT	PIÈGE PRINCIPAL
MMLU-Pro	Connaissances générales + raisonnement	QCM 10 options, ~12k questions	MMLU v1 est saturé et contaminé
GPQA Diamond	Sciences niveau doctorat	QCM expert, 198 questions	Set minuscule : plusieurs points d'écart entre deux runs
AIME	Maths de compétition, raisonnement long	Réponse exacte, 30 problèmes	Un problème = 3,3 points : très instable
SWE-bench Verified	Agentique réelle : corriger un vrai repo	Issues GitHub, tests comme oracle	Mesure autant l'outillage que le modèle
LiveCodeBench	Code, sur des problèmes récents	Fenêtre temporelle glissante	Conçu contre la contamination : c'est son intérêt

Comment un chiffre peut mentir

Contamination

Les questions de l'examen se trouvaient dans les données d'entraînement — comme un élève qui aurait vu le sujet à l'avance. C'est pour ça que des benchmarks datés (LiveCodeBench) existent.

Saturation

Au-dessus de ~90 %, tous les bons modèles ont la même note : l'examen ne discrimine plus rien. Un écart de 1 point n'a alors aucun sens.

Auto-déclaration

Les chiffres d'une model card sont produits par l'auteur du modèle : du marketing jusqu'à reproduction indépendante.

Méthode de comptage

pass@1 (une tentative) et **cons@64** (64 tentatives, réponse majoritaire) ne se comparent pas. Ni deux runs à des températures différentes, ni deux parseurs de réponse différents.

Ce qu'on en fait, concrètement

Le lien direct avec la suite du cours : une **quantization dégrade certains benchmarks et pas d'autres** . Typiquement, les maths et le raisonnement long s'effondrent bien avant les QCM de connaissances. Un score MMLU resté stable ne prouve donc rien sur votre usage.

Construisez un eval maison de 30 exemples représentatifs de votre use case.

1. 30 entrées réelles issues de votre application, avec la sortie attendue.
2. Un critère de réussite explicite par exemple — même binaire, même imparfait.
3. On le relance à chaque changement de modèle, de quant ou de flag.

C'est plus prédictif que n'importe quel leaderboard, ça coûte une demi-journée, et ça resservira à chaque module de ce cours.

Erreurs classiques

Confondre le repo du modèle et un repo de quant tiers

La model card de référence, les recommandations de sampling et la licence sont côté officiel ; le fichier que vous téléchargez est souvent côté tiers.

Prendre un modèle *base* pour un modèle *chat*

Il ne répondra pas aux instructions, et rien dans les logs ne le signalera. Vérifiez le suffixe du nom et la model card.

Croire le context length annoncé

128k annoncés ne veulent pas dire 128k utilisables : il faut que l'engine puisse allouer le KV cache correspondant. C'est exactement le sujet du module suivant.

PROCHAINE ÉTAPE

Vous avez choisi un modèle. Est-ce qu'il rentre en mémoire ?

Module 3 — Budget mémoire et KV cache : les quatre consommateurs de VRAM, le calcul du KV cache sur un vrai `config.json`, et pourquoi la RAM est dix fois plus lente que la VRAM.

03

Budget mémoire et KV cache

Qui consomme la VRAM

Compter les poids ne suffit pas. Quatre postes se cumulent, et un seul grossit pendant l'utilisation.

POSTE	DE QUOI ÇA DÉPEND	ORDRE DE GRANDEUR
Poids	Nombre de paramètres × octets par paramètre	Fixe : 4 à 140 GB
KV cache	Longueur de conversation × nombre de clients simultanés	Variable : de 0,2 GB à plus que les poids
Activations	Taille du batch traité en une passe	Transitoire : quelques centaines de MB
Overhead	Contexte CUDA, buffers de l'engine, fragmentation	0,5 à 1,5 GB, incompressible

S'ajoutent, le cas échéant : un draft model pour le speculative decoding (module 4) et l'encodeur d'images d'un VLM.

Pourquoi le KV cache existe

Sans cache

Pour écrire le 1000^e mot, le modèle devrait relire et **recalculer** les 999 précédents. Puis tout refaire pour le 1001^e. Le coût explose avec la longueur.

C'est un **échange : du calcul contre de la mémoire**. Le cache est obligatoire — aucun engine ne s'en passe — et c'est lui qui consomme la VRAM restante après les poids.

D'où le nom : K pour *keys*, V pour *values* — deux vecteurs stockés par token, par layer, par tête d'attention. C'est exactement ce que dit la formule suivante.

Avec cache

Le travail déjà fait sur les mots passés est **gardé en mémoire** (les vecteurs K et V de chaque token). Chaque nouveau mot ne coûte qu'une passe, pas mille.

Le calcul, sur un modèle réel

$KV = 2 \times layers \times kv_heads \times head_dim \times octets \times tokens$

Sur le config.json vu au module 2 — 28 layers, 4 kv_heads, head_dim 128, KV en fp16 (2 octets) :

$2 \times 28 \times 4 \times 128 \times 2 = 57 \text{ KB par token} \rightarrow \times 4\,096 \text{ tokens} = 0,23 \text{ GB}$

CONTEXTE	KV CACHE, 1 CLIENT	À COMPARER AUX POIDS (8B EN Q4 ≈ 4,5 GB)
4 096 tokens	0,23 GB	5 % des poids — négligeable
32 768 tokens	1,9 GB	40 % des poids — ça compte
131 072 tokens	7,5 GB	Plus que les poids eux-mêmes

Et maintenant, plusieurs clients

Chaque conversation a **son propre** KV cache : il n'y a rien à partager entre deux utilisateurs qui parlent de choses différentes. Le coût se multiplie donc par le nombre de clients simultanés.

SITUATION	KV CACHE TOTAL	SUR UNE CARTE 24 GB, APRÈS 4,5 GB DE POIDS
1 client à 4k	0,23 GB	Large : il reste ~18 GB
32 clients à 4k	7,5 GB	Ça tient encore, tout juste confortable
32 clients à 32k	60 GB	Impossible : il faut réduire l'un des deux

Le réflexe à installer : « le modèle rentre » ne veut rien dire tant qu'on n'a pas dit à *quel contexte* et *pour combien de clients* . C'est exactement l'erreur qui fait échouer un démarrage de vLLM (module 9).

Les cinq leviers de réduction

Reprenez la formule : chaque levier attaque un de ses facteurs. Les quatre premiers sont des choix d'architecture, faits par l'auteur du modèle ; le dernier est un réglage que *vous* contrôlez.

LEVIER	FACTEUR ATTAQUÉ	GAIN TYPIQUE
GQA	kv_heads — des têtes partagent un cache	÷4 à ÷8 (÷7 dans notre exemple)
MQA	kv_heads = 1, le cas limite	Maximum, au prix de la qualité
MLA	head_dim — cache compressé avant stockage	Un ordre de grandeur
Sliding window	tokens — borné par la fenêtre, pas la conversation	Cache plafonné, quelle que soit la longueur
KV quantisé (fp8)	octets — 1 au lieu de 2 par valeur	÷2, coût qualité modéré (module 5)

Sans GQA, le modèle de notre exemple demanderait plus de 50 GB de KV cache à 128k au lieu de 7,5. C'est cette seule ligne de `config.json` qui rend le long contexte praticable.

Où vivent les poids, et à quelle vitesse

La capacité dit ce qui *rentre* ; la bande passante dit à quelle vitesse on peut *lire* . Pour générer un seul token, il faut relire tous les poids actifs : c'est la colonne de droite qui fixe la vitesse (module 4).

MÉMOIRE	CAPACITÉ TYPIQUE	BANDE PASSANTE
HBM — H100 / H200	80–141 GB	3–4,8 TB/s
GDDR — RTX 4090 / 5090	24–32 GB	~1 TB/s
Unified memory — Mac M3/M4 Max	36–128 GB	300–800 GB/s
DDR5 — la RAM du CPU	32–512 GB	50–100 GB/s
NVMe — le disque	Plusieurs To	5–14 GB/s
PCIe 4.0 x16 — le lien GPU–CPU	—	~32 GB/s — le goulot de l'offload

Le rapport à retenir : ~10× entre VRAM et RAM. Un modèle déplacé en RAM ne devient pas un peu plus lent : il devient environ dix fois plus lent. Tout le module 5 tourne autour de cette contrainte.

Vérifier, au lieu d'espérer

Ce que dit le système

`nvidia-smi` — VRAM utilisée / totale, en temps réel. `nv top` pour la version graphique, `rocm-smi` côté AMD.

Attention : une VRAM « pleine » est normale avec vLLM, qui préempte la mémoire d'avance.

Les calculateurs de VRAM sur les Spaces Hugging Face sont pratiques pour un ordre de grandeur, jamais pour une décision : ils ignorent l'overhead de votre engine et de votre driver.

Ce que disent les logs

llama.cpp : la répartition des tensors par device, et la taille allouée du KV cache.

vLLM : le nombre de blocs KV disponibles et la **concurrence maximale estimée** — le chiffre le plus utile de tout le démarrage.

Exercice : le budget de votre machine

Crayon en main, sans lancer quoi que ce soit. Prenez votre propre GPU et un modèle 8B en Q4.

1. Combien de VRAM réellement disponible ?

Capacité de la carte, moins l'affichage du système, moins ~1 GB d'overhead d'engine.

2. Combien pour les poids ?

params × 0,5 octet en Q4. Notez ce qu'il reste.

3. Quel contexte tient dans le reste ?

Divisez la VRAM restante par le coût par token (57 KB dans notre exemple). Puis refaites le calcul pour 4 clients simultanés.

4. Vérifiez

Lancez le serveur avec ce contexte et comparez au KV cache qu'il annonce dans ses logs. Écart ? Cherchez pourquoi.

À MARTELER

« Trois nombres à savoir estimer de tête avant de lancer quoi que ce soit. »

La taille des poids

params × octets par paramètre

Le KV cache visé

au contexte et à la concurrence prévus

La VRAM réellement libre

mesurée, pas celle de la fiche technique

04

Prefill, decode, batching, caching

Comment naît un token

À chaque nouveau mot, le texte traverse le modèle de haut en bas. Cinq étapes, toujours les mêmes :

Embedding Chaque token devient un vecteur de nombres.	N blocs : attention + FFN L'attention regarde le passé (via le KV cache) ; le FFN transforme. Répété 28 à 80 fois.	LM head Un score par mot du vocabulaire : les <i>logits</i> .	Sampling On tire un mot parmi les scores (module 6).
---	--	---	--

Recalculé à chaque token

Toute la traversée pour le *nouveau* token : il faut relire les poids de tous les blocs, une fois.

Gardé en cache

Le travail d'attention sur les tokens passés — le KV cache du module 3. Sans lui, chaque token coûterait toute la séquence.

Deux phases, deux goulots

Une requête se déroule en deux temps qui n'ont rien en commun. Le premier explique les 3 secondes d'attente, le second les 40 tokens/s.

	PREFILL — LIRE LE PROMPT	DECODE — ÉCRIRE LA RÉPONSE
Comment	Tous les tokens du prompt en parallèle , en une passe	Un token à la fois , chacun dépendant du précédent
Ce qui limite	Le calcul : les unités du GPU saturent	La bande passante : relire tous les poids pour un seul token
Coût avec la longueur	Quadratique sur le prompt (l'attention compare chaque token à tous les autres)	Quasi constant par token — les poids pèsent toujours pareil
Métrique	TTFT — time to first token	tokens/s (ou ITL, l'écart entre deux tokens)
Ce qui aide	Plus de compute, meilleurs kernels... ou ne pas le refaire (caching)	Moins d'octets à lire : quantization, ou plus de bande passante

Image : le prefill, c'est lire un dossier — on peut parcourir toutes les pages en même temps, et ça occupe toute l'attention. Le decode, c'est écrire la réponse — un mot après l'autre, sans raccourci possible.

La formule qui prédit la vitesse

tokens/s (1 client) $\approx$ bande passante mémoire $\div$ taille des poids actifs

Pour produire un token en decode, il faut relire tous les poids une fois. Le plafond de vitesse est donc simplement : combien de fois par seconde la mémoire peut-elle livrer le modèle entier ?

SITUATION	CALCUL	PLAFOND
7B bf16 sur RTX 4090	1 000 GB/s $\div$ 14 GB	~70 tok/s
Même 7B en Q4	1 000 GB/s $\div$ 4 GB	~250 tok/s
Même 7B Q4, offloadé en RAM	60 GB/s $\div$ 4 GB	~15 tok/s
MoE 30B-A3B en Q4	1 000 GB/s $\div$ 1,7 GB <i>actifs</i>	~600 tok/s

C'est ça, et rien d'autre, qui justifie la quantization : elle ne réduit pas le nombre d'opérations, elle réduit les octets à déplacer. Et sur un MoE, seuls les paramètres *actifs* entrent dans la formule — d'où leur vitesse.

Batching : dix clients pour le prix d'un

En decode, le GPU relit 4 GB de poids pour produire **un** token et ses unités de calcul s'ennuient. Autant produire dix tokens d'un coup — un par client. Les poids sont lus une fois et servent à tous : le throughput total grimpe, la vitesse de chacun ne bouge presque pas.

Static batching

On attend d'avoir 8 requêtes, on les traite ensemble, on attend que la *plus longue* finisse. Les courtes patientent pour rien.

Continuous batching

Dès qu'une réponse se termine, sa place est reprise par la requête suivante, entre deux tokens. Aucun siège vide. C'est ce que fait vLLM.

Chunked prefill

Un prompt de 30 000 tokens arrive : son prefill est découpé en morceaux pour ne pas geler les réponses en cours des autres clients.

À retenir : le batching augmente massivement le throughput *sans* améliorer la latence individuelle. Et quand prefill et decode se gênent trop, on les sépare sur des GPU distincts : c'est le *disaggregated serving* des grands déploiements.

Ne pas refaire ce qu'on a déjà fait

Un cache évite un calcul déjà fait. Aucun ne change la sortie : ils changent le coût. C'est la seule famille d'optimisations vraiment gratuite du cours.

CACHE	CE QU'IL ÉVITE	OÙ
KV cache	Recalculer l'attention sur les tokens passés	Partout, obligatoire
PagedAttention	Le gaspillage de VRAM par fragmentation : le KV cache est stocké en petits blocs, comme la mémoire virtuelle d'un OS	vLLM, SGLang
Prefix caching	Re-remplir un début de prompt déjà vu : system prompt, exemples, documents	vLLM --enable-prefix-caching , llama.cpp --cache-reuse
Prompt caching (API)	La même idée, facturée moins cher par les providers	Anthropic, OpenAI, Gemini
CUDA graphs, compilation	L'overhead de lancement des kernels, la recompilation au démarrage	vLLM, llama.cpp

Le levier le plus rentable : un system prompt de 2 000 tokens re-remplé à chaque tour de conversation est du gâchis pur. Règle de design qui en découle : le contenu **stable en début** de prompt, le **variable à la fin**.

Speculative decoding : deviner, puis vérifier

Le decode gaspille du calcul : le GPU attend la mémoire. Idée : utiliser ce calcul inoccupé pour vérifier plusieurs mots d'un coup au lieu d'en produire un seul.



La garantie : la sortie est **identique** à celle du decode normal. Ce n'est pas une approximation, c'est le même texte, obtenu plus vite.

Le chiffre qui prédit le gain : l'*acceptance length* , le nombre moyen de tokens gagnés par round. Exemple réel de logs vLLM : 15 proposés, 40 % acceptés, ~7 gagnés par round → ~4× plus rapide pour un client seul.

Qui devine, et quand ça marche

FAMILLE	LE DRAFTER	QUAND LE CHOISIR
Draft model	Un petit modèle de la même famille (un 1B pour un 8B)	Le classique ; llama.cpp -md
N-gram / prompt lookup	Aucun : on recopie des bouts du prompt	Gratuit, excellent en résumé, édition, RAG
Têtes intégrées — EAGLE, MTP	Un mini-réseau greffé sur le modèle, parfois entraîné avec lui (DeepSeek, GLM)	L'état de l'art ; vLLM --speculative-config
Bloc entier — DFlash	Un petit modèle de diffusion qui propose tout le bloc en une passe	Récent ; gains rapportés supérieurs à EAGLE

La limite à connaître absolument : le speculative decoding aide **surtout à faible concurrence** . À forte charge, le GPU est déjà saturé par le batching : le calcul « gratuit » n'existe plus, et la spéculation peut *dégrader* le throughput.

Le nombre de tokens spéculés optimal dépend du modèle, du drafter, du workload et du hardware. À traiter comme une optimisation dépendante de la charge, jamais comme un réglage universel.

Trois « attentions » qu'on confond

FlashAttention

Une façon plus astucieuse de faire **le même calcul**, par petits carreaux, sans jamais écrire la grosse matrice intermédiaire. Plus rapide, moins de mémoire, **zéro perte de qualité**. C'est du calcul.

Vous croiserez aussi FlashInfer, xFormers, Triton, et des kernels dédiés au MLA : des bibliothèques de calcul interchangeables. Le réflexe : un « kernel » change la vitesse, jamais le résultat.

PagedAttention

Une façon de **ranger le KV cache** par blocs, comme la mémoire virtuelle d'un OS, pour ne pas gaspiller de VRAM entre les clients. Rien à voir avec les maths. C'est de la gestion mémoire.

Flash-decoding

FlashAttention adapté au cas d'un seul client avec un très long contexte : on parallélise sur la longueur de la séquence au lieu du batch. C'est du calcul, pour un cas précis.

À MARTELER

« Prefill = calcul = TTFT. Decode = bande passante = tokens/s. Toute optimisation attaque l'un ou l'autre, jamais les deux de la même façon. »

Attaque le prefill

Prefix caching, chunked prefill, FlashAttention, plus de GPU

Attaque le decode

Quantization, speculative decoding, batching, mémoire plus rapide

05

Compression : quantization, pruning, sparsité

Un paramètre, c'est un nombre : sur combien de bits ?

Chaque poids est un nombre à virgule. Le stocker sur moins de bits, c'est le mesurer avec une règle moins fine : 3,14159 devient 3,1 puis 3. On perd de la précision, on gagne de la place — et donc de la vitesse (module 4).

TYPE	BITS	COMPOSITION	USAGE
fp32	32	8 bits d'exposant, 23 de mantisse	Training historique. Jamais en serving.
bf16	16	8 bits d'exposant, 7 de mantisse : même plage que fp32, moins fin	Le format de publication des poids
fp8	8	E4M3 : 4 bits d'exposant, 3 de mantisse	Le défaut en production sur Hopper+
int8	8	Entiers de –128 à 127, plus une échelle	Quasi sans perte
int4 / fp4	4	16 valeurs possibles seulement	Le standard du serving local

Exposant = la *plage* de valeurs représentables (du très petit au très grand) ; mantisse = la *finesse* . bf16 garde la plage de fp32 et sacrifie la finesse : c'est pour ça qu'il a gagné.

Pourquoi compresser, si on décompresse à chaque token ?

Un GGUF Q4 stocke ses poids en 4 bits, mais le GPU ne sait multiplier qu'en 16 bits : chaque poids est donc **décompressé à la volée** , à chaque token, juste avant d'être utilisé. Travail supplémentaire, même nombre de multiplications. Où est le gain ?

Rappel du module 4

En decode, pour chaque token, le GPU doit **relire tous les poids** depuis la mémoire. Les unités de calcul finissent leur travail bien avant que les octets suivants arrivent : elles **attendent**.
tokens/s $\approx$ bande passante $\div$ octets à lire. Le calcul n'apparaît même pas dans la formule.

Donc

Stocker en 4 bits = **4× moins d'octets à lire** par token. La mémoire livre le modèle 4× plus vite.
Décompresser coûte du calcul — mais du calcul, on en avait **en trop** . On dépense une ressource inoccupée pour économiser la ressource saturée.

7B bf16 : 14 GB à lire $\rightarrow$ **~70 tok/s** 7B Q4 + décompression : 4 GB à lire $\rightarrow$ **~250 tok/s**

La limite de la ruse : elle ne marche que tant que le calcul est inoccupé — c'est-à-dire seul sur son GPU. Dès qu'on sert beaucoup de clients en batch, le calcul devient le goulot, et la décompression devient un vrai coût. C'est là que FP8 entre en scène.

Comment on range un nombre sur 4 bits

valeur réelle $\approx$ échelle $\times$ (entier stocké – décalage)

L'exemple simple

Un groupe de 128 poids, tous entre -1 et $+1$. Sur 4 bits on a 16 paliers : échelle = $2/15 \approx 0,13$. Chaque poids est arrondi au palier le plus proche : erreur maximale $\approx 0,07$. Acceptable.

Granularité : une échelle par tenseur entier est trop grossière ; une par groupe de 128 (`group_size=128`) isole les outliers. Plus fin = plus précis, mais plus de métadonnées à stocker.

Le problème des outliers

Le même groupe, mais **un** poids vaut 50. L'échelle devient $100/15 \approx 6,7$: tous les autres poids, entre -1 et $+1$, tombent sur le *même* palier. Un seul nombre a détruit l'information des 127 autres.

Jamais quantisé, ou en dernier : les embeddings, la LM head, les normes, et le router d'un MoE — petits en taille, catastrophiques s'ils sont dégradés.

Deux stratégies : Q4 décompressé ou FP8 natif

La différence n'est pas le nombre de bits. C'est **où se fait le calcul** : en 16 bits après décompression, ou directement en basse précision parce que le GPU sait le faire.

	GGUF Q4, AWQ, GPTQ — « W4A16 »	FP8 — « W8A8 »
Stockage des poids	4 bits	8 bits
Le calcul se fait en	16 bits, après décompression de chaque poids	8 bits, directement : aucune décompression
Octets à lire (le decode)	÷4 — le plus fort gain	÷2
Vitesse de calcul (le prefill, le batch)	Inchangée, voire un peu pire (le coût de décompression)	×2 — les tensor cores calculent en 8 bits
Hardware	N'importe quel GPU, CPU, Mac	Hopper (H100) et plus récent
Qualité	Légère perte (4 bits)	Quasi nulle (8 bits)

Seul sur votre GPU : le goulot est la mémoire. Ce qui compte, c'est la ligne « octets à lire » : **Q4** gagne, et le coût de décompression est invisible.

En production, 50 clients en batch : le goulot devient le calcul. Ce qui compte, c'est la ligne « vitesse de calcul » : **FP8** gagne, et sans perte de qualité.

Arrondir bêtement, ou arrondir en regardant

Toutes ces méthodes sont du *post-training* : on part des poids publiés, on ne réentraîne rien. La différence est dans la façon de choisir les arrondis.

Sans calibration — round-to-nearest

Chaque poids est arrondi au palier le plus proche, indépendamment des autres.

Rapide, simple, aveugle aux outliers.

bitsandbytes (`load_in_4bit`) — très utilisé en développement, lent au serving. GGUF legacy : Q4_0, Q8_0.

Avec calibration

On fait passer quelques centaines de vrais textes dans le modèle, on observe quels poids comptent vraiment, et on choisit les arrondis pour **minimiser l'erreur en sortie** , pas l'erreur sur chaque poids.

GPTQ — couche par couche, corrige les arrondis suivants avec l'erreur des précédents. **AWQ** — *activation-aware* : repère les canaux saillants et les protège.

SmoothQuant — déplace la difficulté des activations vers les poids, prérequis du W8A8.

À reconnaître sans creuser : HQQ, QuIP#, AQLM, EXL2/EXL3, AutoRound. Et le **QAT** (quantization-aware training) : quand l'auteur entraîne le modèle en sachant qu'il sera quantisé (Gemma). Bat toujours le post-training à bitrate égal — mais ce n'est pas vous qui le faites.

Une échelle pour combien de poids ?

Rappel : un poids quantisé = un petit entier × une **échelle**. L'échelle est un vrai nombre en 16 bits qu'il faut stocker aussi. Toute la question est : *combien de poids partagent la même échelle ?*

Une échelle pour tout le tenseur

Quasi gratuit en place. Mais un seul outlier parmi des millions de poids écrase tout le monde (slide 64).
Trop grossier.

Une échelle par bloc de 32 poids

Un outlier n'abîme que ses 31 voisins. C'est ce que font tous les formats modernes. Coût : 16 bits d'échelle pour 32 poids, soit **+0,5 bit par poids** . Un « Q4 » pèse donc 4,5 bits.

Une échelle par poids

Parfait... et absurde : l'échelle pèse plus que le poids qu'elle décrit. On n'a rien compressé.

L'astuce de la super-échelle

Les échelles elles-mêmes se ressemblent d'un bloc à l'autre. Alors on les quantise aussi : chaque bloc garde une **mini-échelle sur 6 bits** , et un groupe de 8 blocs (256 poids) partage une seule **super-échelle** en 16 bits. On garde la finesse des petits blocs pour presque rien : c'est exactement l'idée du « K » des k-quants.

À retenir : quand vous lisez « blocs de 32 », « group_size 128 » ou « microscaling », c'est toujours la même question — la taille du groupe qui partage une échelle. Plus petit = plus précis = un peu plus lourd.

Lire un nom de fichier GGUF

Q4_K_M

Q4 — environ 4 bits par poids en moyenne. Q2 à Q8 existent.

K — la méthode de la slide précédente : petits blocs, mini-échelles, super-échelle. Sans le K (Q4_0), une seule grosse échelle par bloc : plus vieux, moins bon.

M — le *mix* . Tous les tenseurs ne sont pas quantisés pareil : les plus sensibles (attention, sortie) reçoivent 5 ou 6 bits, les autres 4. S = moins de tenseurs protégés, L = plus.

NIVEAU	TAILLE POUR UN 8B	VERDICT
Q8_0	~8,5 GB	Quasi identique au bf16. Si la VRAM le permet.
Q6_K / Q5_K_M	~6,5 / 5,7 GB	Excellent compromis
Q4_K_M	~4,9 GB	Le défaut raisonnable
Q3_K_M et en dessous	~4 GB et moins	Dégradation visible. En dessous, préférer les IQ (slide suivante).

La taille ne descend pas linéairement avec les bits : les échelles, les tenseurs protégés par le mix et les embeddings (jamais quantisés) forment un plancher.

Deux mots que vous verrez sur chaque repo GGUF

IQ — les i-quants (IQ2_XS, IQ4_XS...)

Un k-quant range chaque poids sur des **paliers réguliers** : 16 valeurs également espacées pour 4 bits. Or les poids d'un réseau ne sont pas répartis uniformément : beaucoup près de zéro, très peu aux extrêmes. Des paliers réguliers gaspillent des valeurs là où il n'y a presque rien.

Un i-quant utilise à la place un **dictionnaire de valeurs typiques** (un *codebook*), dense près de zéro, clairsemé aux extrêmes. Comme une image GIF : 256 couleurs choisies pour *cette* photo, pas 256 couleurs régulières.

Résultat : nettement meilleur que les k-quants à 2 et 3 bits. Un peu plus lent au decode (il faut consulter le dictionnaire). Inutile à 4 bits et plus.

Les deux se combinent : les IQ sont quasi toujours produits avec imatrix, et les meilleurs quantizers l'appliquent aussi à leurs k-quants. Quand un repo indique « imatrix » et documente son corpus, c'est bon signe.

imatrix — l'importance matrix

Ce n'est **pas un format**, c'est une méthode de fabrication. Avant de quantiser, on fait passer quelques centaines de textes dans le modèle et on mesure, pour chaque poids, **à quel point il influence la sortie**.

Ensuite, quand on choisit les arrondis d'un bloc, on accepte une grosse erreur sur les poids qui ne comptent pas, pour garder une petite erreur sur ceux qui comptent. Même nombre de bits, erreur mieux placée.

À bitrate égal, un quant *avec* imatrix bat toujours un quant sans. **Le piège** : « ce qui compte » a été mesuré sur le corpus de calibration. Calibré en anglais seulement, le quant protège les poids utiles à l'anglais — et dégrade un peu plus le français.

Les formats que le GPU exécute directement

Les formats de la slide précédente côté « FP8 » : ceux que les tensor cores récents savent multiplier **directement** , sans décompression. Leur intérêt dépend entièrement du GPU que vous avez.

FORMAT	EN CLAIR	GPU QUI LE CALCULE EN NATIF	QUAND VOUS LE CROISEREZ
FP8	Des nombres à virgule sur 8 bits — le bf16 coupé en deux	H100 et plus récent	Le choix par défaut en production : quasi sans perte, 2× moins de mémoire, 2× plus de calcul
MXFP4	Des nombres à virgule sur 4 bits, une échelle tous les 32 poids — les blocs de la slide 67, mais compris directement par le GPU	RTX 50 / B200 ; émulé (donc décompressé) sur les autres	gpt-oss d'OpenAI est publié directement dans ce format
NVFP4	La variante NVIDIA : une échelle tous les 16 poids, un peu plus précis	RTX 50 / B200 uniquement	Les quants « NVFP4 » publiés par NVIDIA et RedHatAI
INT8 / INT4	Des entiers, pas des nombres à virgule, avec une échelle	Depuis les RTX 30 (int8)	Les repos « W8A8 » ou « compressed-tensors » pour vLLM

« MX » = un standard commun à NVIDIA, AMD et Intel. Conséquence : **votre GPU décide** . Un FP8 sur une RTX 3090 sera décompressé comme un GGUF, sans gain de calcul ; un NVFP4 exige du Blackwell.

Le tableau à graver : quel format pour quel engine

FORMAT	LLAMA.CPP	VLLM / SGLANG	MLX (MAC)	NOTE
GGUF	Natif	Partiel, expérimental	Non	Mono-fichier, tokenizer inclus
safetensors bf16	Conversion requise	Natif	Conversion	Le point de départ de tout
AWQ / GPTQ	Non	Oui	Non	W4A16 côté GPU
FP8 / compressed-tensors	Non	Oui	Non	Hopper+
MXFP4 / NVFP4	Partiel	Oui	Non	Blackwell pour NVFP4
MLX 4/8-bit	Non	Non	Natif	Apple Silicon uniquement

Le choix du format de quantization détermine le choix de l'engine, et réciproquement. Ce n'est pas une décision indépendante.

Outils pour produire ces formats : llama-quantize pour le GGUF, llm-compressor (Red Hat) pour vLLM, AutoAWQ / GPTQModel, mlx_lm.convert pour Apple.

Deux réflexes de plus

Juger un quant téléchargé chez un tiers

Une imatrix a-t-elle été utilisée , et sur quel corpus ? Non documenté = présumer que non.

Une mesure est-elle reportée ? Perplexité ou KL-divergence par rapport au bf16 : c'est le minimum.

Les tailles de fichiers sont-elles cohérentes avec le calcul $\text{params} \times \text{bits}$? Un Q4 de 8B qui fait 3 GB cache quelque chose.

Le quantizer est-il connu ? unsloth, bartowski, mradermacher, RedHatAI documentent leur méthode.

Quantiser aussi le KV cache

Au module 3, le KV cache dépassait les poids à 128k de contexte. En **fp8** il est divisé par deux, pour un coût qualité modéré. En long contexte, c'est souvent **le meilleur levier** disponible.

llama.cpp : `--cache-type-k q8_0 --cache-type-v q8_0` — K tolère mieux l'agressivité que V. vLLM : `--kv-cache-dtype fp8`.

En dessous de 8 bits, la dégradation devient réelle et *augmente* avec la longueur — exactement là où on en aurait le plus besoin.

Pruning : enlever des morceaux, pas des bits

La quantization attaque la redondance **numérique** (trop de précision par poids). Le pruning attaque la redondance **structurelle** (des poids, des têtes, des blocs entiers qui servent peu). Deux axes indépendants, combinables.

Le cas d'école : REAP, supprimer des experts d'un MoE

Dans un MoE, le router n'appelle pas tous les experts à la même fréquence. REAP mesure, sur du vrai texte, **à quel point chaque expert est sollicité** et **combien il pèse** dans la réponse — puis supprime les moins utiles. Sans réentraînement.

Pourquoi supprimer plutôt que fusionner : fusionner deux experts détruit la capacité du router à choisir finement. Supprimer préserve ce contrôle. Sur les tâches génératives, le pruning bat le merging.

Exemple : Qwen3-Coder-480B-A35B → REAP-246B-A35B : **50 % des experts supprimés**, ~97 % des performances en code conservées. La mémoire est divisée par deux ; la vitesse ne bouge pas (A35B est inchangé).

Autres formes, à reconnaître : supprimer des blocs entiers (depth pruning), des têtes d'attention, ou imposer 2 zéros sur 4 poids (sparsité 2:4, accélérée par les tensor cores). SparseGPT, Wanda. La distillation est l'alternative : hors périmètre.

La ruse des experts en RAM

Troisième famille : ne rien enlever, mais exploiter le fait qu'une partie du modèle **dort** la plupart du temps. Le raisonnement, en trois étapes :

1. L'attention

Tourne à **chaque** token et est **petite** . Elle doit rester en VRAM.

2. Les experts

Énormes, et chacun **rarement** sollicité. Ils peuvent vivre en RAM.

3. Donc

On n'offload pas des *layers* : on offload les *tenseurs d'experts*. C'est `--n-cpu-moe` dans llama.cpp.

Pourquoi ça ne marche pas sur un dense 70B : chaque paramètre sert à chaque token. Il n'y a pas de masse dormante à exiler — tout ce qu'on mettrait en RAM serait relu à chaque token, à 60 GB/s.

La courbe en V : trop peu d'offload = la VRAM déborde et le driver pagine en silence, catastrophique. L'optimum est la *plus petite* valeur qui tient vraiment ; au-delà, déclin régulier. **Si augmenter l'offload vous accélère, c'est le symptôme que quelque chose était déjà cassé.**

À MARTELER

« Trois questions, dans cet ordre. »

1. Quel bitrate mon hardware supporte-t-il ?

FP8 exige Hopper, NVFP4 exige Blackwell. Le W4A16 tourne partout.

2. Quel format mon engine accepte-t-il ?

GGUF pour llama.cpp, safetensors / AWQ / FP8 pour vLLM. Le tableau à graver.

3. Ce quant a-t-il été produit sérieusement ?

imatrix, mesure reportée, tailles cohérentes, quantizer connu.

06

Génération et sampling

Le modèle ne choisit pas un mot : il note tous les mots

À chaque étape, la sortie du modèle est un score par mot du vocabulaire — 150 000 nombres. Le *sampling* , c'est la recette qui transforme ces scores en un seul mot choisi. Elle est exécutée par le serveur, pas par le modèle.



Exemple : après « La capitale de la France est », le modèle note *Paris* très haut, *Lyon* bas, *banane* très bas. À température 0 on prend *Paris* . À température 1, *Lyon* sortira parfois. Sans troncature, *banane* sortira un jour.

L'ordre compte : appliquer la température avant ou après la troncature ne donne pas le même résultat — et l'ordre *diffère* entre llama.cpp et vLLM. Mêmes paramètres, sorties différentes.

Les boutons, et ce qu'ils font

PARAMÈTRE	EFFET	VALEURS COURANTES
temperature	Divise les scores : <1 concentre sur les favoris, >1 aplatit vers l'uniforme. 0 = toujours le premier (greedy).	0,6–0,8 chat ; 0 extraction
top_k	Ne garde que les k mots les mieux notés	20–50
top_p	Garde les mots jusqu'à ce que leur probabilité cumulée atteigne p	0,9–0,95
min_p	Garde les mots dont la probabilité vaut au moins p × celle du favori. Plus robuste que top_p à haute température.	0,05–0,1
repeat_penalty	Baisse le score des mots déjà apparus dans les N derniers tokens (llama.cpp)	1,0–1,1 ; trop haut = charabia
presence / frequency_penalty	Même idée, style OpenAI : forfait par mot vu, ou proportionnel à sa fréquence	0–0,5
stop, max_tokens, seed, logit_bias	Contrôle : où s'arrêter, combien au maximum, fixer le hasard, favoriser ou interdire un mot	Selon l'usage

À reconnaître sans creuser : **DRY** (pénalise les séquences répétées, pas les mots), **XTC** (exclut parfois les favoris, pour la créativité), **mirostat** (régule la surprise en boucle fermée), typical_p, top_n_sigma.

Température 0 ne veut pas dire « toujours pareil »

Ce qu'on attend

Greedy = on prend toujours le mot le mieux noté. Pas de hasard, donc même entrée → même sortie. Logique.

Ce qui se passe

Les scores sont calculés en virgule flottante, où $(a+b)+c \neq a+(b+c)$ à la dernière décimale. Le **batching change l'ordre des additions**. Quand deux mots sont presque à égalité, la dernière décimale décide — et la suite diverge.

Conséquences : votre résultat dépend de *qui d'autre* était servi en même temps. `seed` fixe le tirage aléatoire, pas l'ordre des calculs : il ne suffit pas si le batch varie.

Le vrai déterminisme (*batch-invariance*) est un problème ouvert : des kernels dédiés existent, au prix de la vitesse. Pour un eval, la seule protection est de répéter la mesure.

Forcer du JSON valide

Le *constrained decoding* reprend le pipeline : à chaque étape, on met un score de $-\infty$ à tous les mots qui rendraient la sortie invalide selon une grammaire. Le modèle ne **peut** pas se tromper de syntaxe : une accolade ouverte sera fermée.

Les outils

llama.cpp : grammaires GBNF (`--grammar-file`), convertibles depuis un JSON Schema.

vLLM : `response_format`, `guided_json`, `guided_choice` , via xgrammar ou outlines.

Sous le capot : un automate qui suit la grammaire token par token.

L'alternative honnête : demander du JSON dans le prompt, parser, réessayer en cas d'échec. Souvent suffisant, parfois meilleur. Ne sortez la grammaire que si le taux d'échec le justifie.

Les coûts

Compilation de la grammaire au premier appel ; incompatibilités avec certains kernels ou le speculative decoding.

Et surtout : **dégradation de qualité** si la contrainte force le modèle hors de ce qu'il aurait naturellement dit. Un schéma trop rigide donne du JSON parfait au contenu médiocre.

Qui décide vraiment des paramètres ?

Quatre sources de vérité, qui ne s'accordent pas. La dernière qui parle gagne ; les autres sont ignorées sans avertissement.

-
- 1 **Ce que l'auteur du modèle recommande** — `generation_config.json`, model card. La meilleure source, et la moins écoutée.
 - 2 **Ce que l'engine applique par défaut** — et llama.cpp ≠ vLLM. Ni l'un ni l'autre ne lit le fichier de l'auteur.
 - 3 **Ce que le serveur a été lancé avec** — les flags de la ligne de commande. Écrase le n°2.
 - 4 **Ce que la requête envoie** — l'override final, par appel. Écrase tout.
-

Exemple typique : un modèle reasoning documenté pour `temperature=0.6`, `top_p=0.95`, `top_k=20` tourne avec les defaults de l'engine parce que personne ne les a passés. Il boucle, il délire — et on conclut que « ce modèle est nul ».

À MARTELER

« Les recommandations officielles d'un modèle ne sont jamais appliquées automatiquement par les serveurs. »

Avant de déclarer qu'un modèle est mauvais : ouvrir sa model card, lire les paramètres recommandés, les passer explicitement au serveur, et refaire le test. Une fois sur deux, le problème disparaît.

07

llama.cpp

Un binaire, zéro dépendance, tout le hardware

Écrit en C++ au-dessus de la bibliothèque GGML. Pas de Python, pas d'environnement à installer : on télécharge un exécutable, on lui donne un GGUF, ça tourne. C'est ce qui en fait l'outil du poste de travail.

Les backends : où ça tourne

CPU seul (AVX2, AVX-512, NEON sur ARM) — lent mais universel.

CUDA (NVIDIA), **ROCm** (AMD), **Metal** (Mac), **Vulkan** (n'importe quel GPU), SYCL (Intel).

Et surtout : le **mode hybride** GPU + CPU, une partie du modèle sur chacun.
Aucun autre engine ne le fait aussi bien.

L'obtenir : releases précompilées sur GitHub (prendre celle de *votre* backend), image Docker, ou compilation avec `cmake -DGGML_CUDA=ON`. Le fork **ik_llama.cpp** ajoute des quants IQ plus fins et un offload MoE plus granulaire.

Les binaires : lesquels retenir

`llama-server` — le serveur HTTP OpenAI-compatible. Celui qu'on utilise.

`llama-bench` — mesurer prefill et decode (module 8). `llama-quantize` — produire un GGUF.

Aussi : `llama-cli` (chat en terminal), `llama-perplexity`,
`llama-tokenize` .

Groupe A — où vont les octets

Les flags à régler en premier, dans cet ordre. Ils décident si ça rentre et à quelle vitesse ça lit.

FLAG	CE QU'IL FAIT	PIÈGE
-hf repo:Q4_K_M	Télécharge le GGUF depuis Hugging Face, avec choix du quant	Repo gated : HF_TOKEN requis
-ngl 99	Nombre de layers placés sur le GPU. 99 = tout ce qui rentre	Ne pas le baisser pour gérer un MoE : utiliser la ligne suivante
--n-cpu-moe N	Envoie les <i>experts</i> des N premiers layers en RAM, garde l'attention sur GPU (module 5)	Chercher la plus <i>petite</i> valeur qui tient
-c 8192	Taille du contexte réservé — donc du KV cache (module 3)	-c 0 = le max du modèle, 128k parfois : la VRAM explose
-fa on	FlashAttention (module 4)	Quasi toujours à activer
--cache-type-k q8_0	KV cache quantisé (module 5). Idem -v	K tolère mieux que V ; nécessite -fa
-ot "regex=CPU"	Placement tenseur par tenseur, par expression régulière sur leur nom	L'outil de précision, quand --n-cpu-moe ne suffit pas
--split-mode, --tensor-split	Répartition entre plusieurs GPU	Ne pas toucher aux flags --rope-* sans raison

Groupe B et C — le débit, puis le serveur

B — Débit et parallélisme

-t 8	Threads CPU pour le decode : le nombre de cœurs <i>physiques</i>
-ub 512	Micro-batch du prefill : le vrai levier VRAM / vitesse de prefill
-np 4	4 utilisateurs simultanés. Le contexte est divisé : -c 8192 -np 4 = 2048 par personne
--cache-reuse 256	Prefix caching (module 4)
-md draft.gguf	Speculative decoding avec un petit modèle (module 4)
--ctx-shift	Quand le contexte est plein : glisser au lieu d'échouer. Effets qualité

C — Serveur et API

--host --port	Exposition. Jamais 0.0.0.0 sans --api-key
--jinja	Utilise le vrai chat template du GGUF. Obligatoire pour le tool calling (module 10)
--reasoning-format	Sort le <think> dans un champ séparé
--alias nom	Le nom exposé dans /v1/models
--mmproj	Le projecteur d'images pour un VLM
/props /slots /metrics	Introspection : /props dit quel template est <i>vraiment</i> utilisé

Lire les logs : quatre lignes qui expliquent tout

```
ggml_cuda_init: found 1 CUDA devices: NVIDIA GeForce RTX 4090 load_tensors: offloaded 37/37 layers to GPU
load_tensors:  CUDA0 model buffer size = 4685.30 MiB load_tensors:  CPU model buffer size = 292.36 MiB
llama_kv_cache: CUDA0 KV buffer size = 576.00 MiB (n_ctx = 8192, type_k = f16)
common_chat_templates_init: chat template detected:
chatml
```

1. Le backend actif. Si vous lisez « no CUDA devices », tout tourne sur CPU : mauvais binaire ou mauvais driver. Rien d'autre ne compte avant.

3. La taille du KV cache. 576 MB pour 8192 tokens : vous pouvez refaire ce calcul (module 3). S'il dépasse la VRAM restante, réduire -c.

2. La répartition par device. 37/37 layers sur GPU et 4,7 GB en VRAM : le modèle tient. Les 292 MB côté CPU sont normaux (embeddings).

4. Le template détecté. S'il ne correspond pas à la famille du modèle, les réponses seront mauvaises sans aucune erreur (module 10).

Quand « ça ne va pas assez vite », ces quatre lignes se lisent avant de toucher au moindre flag.

Quatre recettes de référence

Petit dense qui tient en VRAM — le cas simple

```
llama-server -hf bartowski/Qwen3-8B-GGUF:Q4_K_M \
  -ngl 99 -c 16384 -fa on --jinja
```

Long contexte — KV cache quantisé

```
llama-server -m model.gguf -ngl 99 -c 65536 -fa on \
  --cache-type-k q8_0 --cache-type-v q8_0 --cache-reuse
```

Pour la recette MoE : partir de `--n-cpu-moe` élevé, baisser jusqu'à ce que la VRAM déborde, remonter d'un cran. La plus petite valeur qui tient est l'optimum (courbe en V, module 5).

Gros MoE avec expert offload — 30B-A3B sur 12 GB

```
llama-server -hf unsloth/Qwen3-30B-A3B-GGUF:Q4_K_M \
  -ngl 99
--n-cpu-moe 20 --cache-type-k q8_0 --cache-type-v q8_0 --jinja
```

Plusieurs utilisateurs — 4 slots de 8k chacun

```
llama-server -m model.gguf -ngl 99 -fa on \
  -c 32768 -np 4 --cache-key secret --cache-type-k q8_0
```

08

Mesurer au lieu de croire

Il n'y a pas « une vitesse »

« Ce serveur fait 80 tokens/s » ne veut rien dire tant qu'on n'a pas dit *pour combien de clients à la fois* . Le même serveur, mesuré à plusieurs niveaux de concurrence :

CLIENTS SIMULTANÉS	TOK/S PAR CLIENT	TOK/S TOTAL (THROUGHPUT)	TTFT	LECTURE
1	80	80	0,3 s	Confort maximal, GPU sous-utilisé
8	65	520	0,6 s	Le batching travaille : 6,5× plus de débit pour –20 % par client
32	30	960	1,8 s	Le GPU sature ; chacun attend plus
64	16	1 020	4,5 s	Plus de débit gagné, seulement de l'attente : la file s'allonge

Ce tableau est un **front de Pareto** : on choisit un point selon ce qu'on optimise. Le poste de travail vit sur la première ligne, la production autour de la troisième.

Goodput : le débit qui respecte une contrainte de latence (ex. TTFT < 2 s). Ici, le goodput plafonne à 32 clients : au-delà, on produit plus mais on ne sert plus correctement.

Ce qu'on mesure, et avec quoi

Les métriques

TTFT	Temps avant le premier token. Dominé par le prefill et la file d'attente
ITL / TPOT	Écart entre deux tokens. L'inverse des tok/s perçus
tok/s par client	Ce que <i>vous</i> voyez défiler
Throughput total	Ce que le GPU produit, tous clients confondus. Ce qui fixe le coût / M tokens
Goodput	Le throughput qui respecte votre objectif de latence

Les outils

llama-bench	Prefill (pp) et decode (tg) en tok/s, pour comparer deux flags ou deux quants
vllm bench serve	La courbe complète de la slide précédente, contre un serveur qui tourne
GuideLLM	Sweeps de charge réalistes, rapport de goodput
/metrics + Grafana	L'observabilité en continu, format Prometheus, sur les deux engines
llama-perplexity, KL-div	La qualité d'un quant, en relatif au bf16 — jamais en absolu

Et pour la qualité sur *votre* tâche, rien ne remplace l'eval maison de 30 exemples du module 2. La perplexité dit si un quant a abîmé le modèle, pas s'il fait encore votre travail.

À MARTELER

« **Changez *un* flag. Mesurez. Notez.** »

Le folklore des forums est massivement faux, non par malhonnêteté mais parce qu'il compare des configurations incomparables : quant différent, contexte différent, concurrence différente, version différente. Un tableur avec une ligne par essai vaut plus que tous les threads Reddit.

09

vLLM

Même API, philosophie opposée

vLLM part de deux hypothèses : le modèle tient en VRAM, et le GPU doit être saturé par des dizaines de clients. Chaque ligne de ce tableau en découle.

	LLAMA.CPP	VLLM
Cible	1 utilisateur, hardware hétérogène	N utilisateurs, GPU datacenter
Formats	GGUF	safetensors, AWQ, GPTQ, FP8, MXFP4 / NVFP4
Si ça ne rentre pas	Offload granulaire en RAM, first-class	Échec au démarrage. L'offload existe mais dégradé
KV cache	Un bloc contigu par slot	PagedAttention : petits blocs partagés entre clients (module 4)
Prefix caching	Opt-in	Automatique et central
Multi-GPU	Découpe par layers	TP / PP / DP / EP, multi-machine
Installation	Un binaire	Python + CUDA, lourd, sensible aux versions
Démarrage	Secondes	Dizaines de secondes à minutes : compilation, CUDA graphs
Force	Flexibilité mémoire	Throughput et coût par token

Les flags qui décident si ça démarre

```
vllm serve Qwen/Qwen3-8B-FP8 --max-model-len 16384 --max-num-seqs 64 --gpu-memory-utilization 0.90
--served-model-name qwen3
```

FLAG	CE QU'IL FAIT	À COMPRENDRE
<code>--gpu-memory-utilization 0.9</code>	La part de VRAM que vLLM réserve d'avance : poids + tout le reste pour le KV cache	Une cible , pas une limite. La VRAM apparaîtra pleine dans <code>nvidia-smi</code> : c'est normal
<code>--max-model-len</code>	Contexte maximal par requête	Par défaut, celui du modèle — souvent 128k, souvent trop
<code>--max-num-seqs</code>	Nombre maximal de requêtes traitées ensemble	Le produit des deux doit tenir dans le KV cache (module 3) : l'échec n°1 au démarrage
<code>--kv-cache-dtype fp8</code>	KV cache quantisé	Double la concurrence possible, quasi gratuit
<code>--quantization, --dtype</code>	Format des poids (souvent détecté automatiquement)	Le repo doit être dans un format accepté (tableau du module 5)
<code>--max-num-batched-tokens</code>	Tokens traités par pas de scheduling	Le curseur TTFT / throughput : plus haut = plus de débit, plus d'attente

Le prefix caching et le chunked prefill sont activés par défaut. La moitié des optimisations réelles vit dans des variables d'environnement `VLLM_*` : source de frustration légitime.

Plusieurs GPU : quatre façons de les faire travailler

Analogie : un livre à traduire, quatre traducteurs.

MODE	L'ANALOGIE	CE QUI SE PASSE	CE QU'ON GAGNE
TP — tensor parallel	Les quatre traduisent <i>chaque phrase</i> ensemble, un quart chacun	Chaque layer est découpé entre GPU ; ils se synchronisent à chaque token	Latence divisée, gros modèle qui rentre. Exige un lien rapide (NVLink)
PP — pipeline parallel	Chacun traduit un quart du livre, à la chaîne	Les layers 1–10 sur GPU 1, 11–20 sur GPU 2...	Ça rentre, tolère un lien lent. Pas plus rapide
DP — data parallel	Quatre copies du livre, quatre lecteurs indépendants	Le modèle complet répliqué sur chaque GPU	Throughput multiplié. Latence inchangée
EP — expert parallel	Chaque traducteur est spécialiste d'un domaine	Les experts d'un MoE répartis entre GPU	Pour les très gros MoE

TP divise la latence, DP multiplie le throughput. Ce ne sont pas des substituts : on combine souvent les deux (`-tp 2 -dp 2` sur 4 GPU).

Lire les logs : le chiffre qui compte

```
INFO Using FLASH_ATTN attention backend
INFO Loading weights took 12.4 seconds
INFO Available KV cache memory:
14.2 GiB INFO GPU KV cache size: 262,144 tokens INFO Maximum concurrency for 16,384 tokens per request: 16.0x INFO
Graph capturing finished in 18 secs
```

KV cache disponible : ce qu'il reste après les poids, dans les 90 % réservés.
14,2 GiB = 262k tokens pour ce modèle — le calcul du module 3, fait par le serveur.

Concurrence maximale : $262k \div 16k = 16$ **requêtes** pleines à la fois. Si vous en attendez 64, réduisez `--max-model-len` ou passez le KV en fp8.

L'erreur n°1 : `ValueError: The model's max seq len (131072) is larger than the maximum number of tokens that can be stored in KV cache` — le modèle tient, mais pas *une seule* requête à son contexte maximal. Solution : `--max-model-len 32768`.

Speculative decoding, et ce qu'on nomme sans creuser

Un seul flag, du JSON

```
--speculative-config '{"method":"ngram",  
  "num_speculative_tokens":4}'  
  
--speculative-config '{"method":"eagle3",  
  
  "model":"yuhuili/EAGLE3-..." ,  
  "num_speculative_tokens":3}'
```

Le détail qui fait perdre une heure : `model` est requis pour `eagle3` et `draft_model`, mais résolu automatiquement pour `mtp` et `ngram`. Et rappel du `module 4` : utile à faible concurrence, contre-productif quand le GPU sature.

À reconnaître dans une conversation

SGLang — l'alternative directe à vLLM, avec RadixAttention pour le prefix caching. Même philosophie, choix différents.

Disaggregated prefill / decode — des GPU dédiés au prefill, d'autres au decode (module 4). Les très grands déploiements.

LMCache, KV connectors — garder le KV cache en RAM ou sur disque entre requêtes plutôt que le jeter.

Multi-nœud avec Ray, production-stack Kubernetes — quand un serveur ne suffit plus.

Aussi : `--tool-call-parser`, `--reasoning-parser`, `--enforce-eager` (désactive les CUDA graphs : debug seulement).

10

Couche client et intégration

Le chat template : première cause de bugs

Au module 1, vous avez vu que vos messages propres deviennent une chaîne balisée (`<|im_start|>user...`). Le **chat template** est le petit programme (Jinja) qui fait cette traduction. Il est propre à chaque famille de modèle, et quand il est faux, **rien ne plante** : le modèle répond juste mal.

Les pathologies

Template absent ou générique — un GGUF converti à la va-vite se retrouve avec le template d'une autre famille.

BOS dupliqué — le token de début ajouté deux fois : qualité dégradée, symétriquement invisible.

Rôle system non supporté — certaines familles l'ignorent ou plantent.

Balises de tool call non gérées — le modèle les produit, le template ne les décrit pas.

Où il vit : `tokenizer_config.json` ou `chat_template.jinja` dans le repo (module 2), embarqué dans les métadonnées du GGUF.

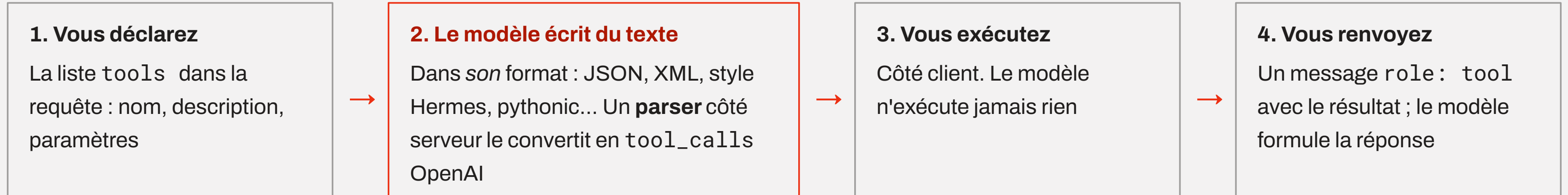
Comment vérifier

llama.cpp : `GET /props` renvoie le template réellement utilisé. Lancer avec `--jinja` pour utiliser celui du GGUF plutôt qu'une détection.

vLLM : les logs de démarrage ; `--chat-template` pour forcer un fichier.

En local : `tokenizer.apply_chat_template(messages, tokenize=False)` et lire le résultat à l'œil. Deux minutes qui évitent une journée.

Tool calling : la boucle complète



llama.cpp

`--jinja` active le template complet du GGUF, qui décrit le format d'appel d'outil. Sans lui, pas de tool calling.

vLLM

`--enable-auto-tool-choice --tool-call-parser hermes` (ou `llama3_json`, `qwen3_coder`...) : le parser doit correspondre à la famille.

Pourquoi « le tool calling ne marche pas » : parser absent ou mal choisi, neuf fois sur dix.

Reasoning, clients, mise en production

Le raisonnement

Un modèle thinking produit un bloc `<think>` avant sa réponse. Sans réglage, il **fuit dans content** : l'utilisateur le voit, votre parseur JSON casse.

```
--reasoning-format (llama.cpp) /  
--reasoning-parser (vLLM) le placent dans  
reasoning_content, à part.
```

Ces tokens se paient et comptent dans `max_tokens` : prévoir large.

Open WebUI et les clients

Une interface de chat complète : multi-modèles, utilisateurs, historique, RAG, recherche web. **C'est un client, pas un engine** — il parle à votre serveur via l'API OpenAI.

Connexion : « OpenAI API » → `http://host:8080/v1` . Pièges : le `/v1` oublié, `host.docker.internal` depuis Docker, timeouts sur les modèles lents.

Alternatives : LibreChat, Jan, AnythingLLM. Pour coder : Continue, Cline, Aider, OpenCode.

Production, en survol

LiteLLM devant plusieurs backends : un seul endpoint, plusieurs engines, routage et quotas.

Docker Compose, et `HF_HOME` sur un disque dédié : le cache de modèles pèse vite des centaines de Go.

Sécurité : port jamais exposé nu, `--api-key` , reverse proxy, rate limiting, logs.

11

L'arbre de décision

Trois questions, dans l'ordre

1. Quel hardware ?

Mac Apple Silicon	MLX ou llama.cpp (Metal), 4-bit
CPU seul	llama.cpp, Q4_K_M, attentes réalistes
1 GPU consumer, 1 utilisateur	llama.cpp + GGUF
1 GPU consumer, plusieurs	vLLM si ça tient en VRAM, sinon llama.cpp -np
GPU datacenter, multi-users	vLLM (ou SGLang), FP8 ou AWQ, prefix caching

2. Ça ne rentre pas

- 1. C'est un MoE ? → expert offload --n-cpu-moe
- 2. KV cache en fp8, puis contexte réduit
- 3. Quant plus agressif : Q5 → Q4_K_M → IQ4_XS — *et mesurer*
- 4. Modèle plus petit — souvent la meilleure réponse
- 5. Multi-GPU : TP=2 ou layer split

3. C'est lent

TTFT élevé → c'est le **prefill** : prefix caching, chunked prefill, -ub, prompt plus court

tok/s faible → c'est le **decode** : bande passante → quantization, ou speculative decoding si peu de clients

Et toujours : lire les logs d'abord, changer un flag à la fois.

FIN DU COURS

Qu'est-ce qui tourne sur ma machine, et pourquoi à cette vitesse-là ?

Vous savez maintenant répondre : quel modèle, dans quel format, sur quel engine, avec quels flags — et comment le vérifier.